



IBN ZOHR UNIVERSITY
FACULTY OF SCIENCES – IT CENTER OF EXCELLENCE

Computer Engineering and Embedded Systems
MASTER IISE

Python Module Project
Report: AI-Based Water
Potability Prediction
Using Machine Learning

This report describes the design, implementation, and testing of a Python-based water potability prediction system.

Project Repository: <https://github.com/uallide/ai-water-potability-prediction>

Prepared by:

Oualid OUARRACH
Mahmoud CHARABARA
Youness EZIDY
Imad ADDIOUI

Supervised by:

Prof. Yassine OUKDACH

Academic Year 2025-2026

Table of Contents

1. Abstract
 2. Introduction
 3. Problem Statement and Objectives
 4. Dataset Description
 5. Tools and Technologies
 6. Project Repository and File Structure
 7. System Architecture
 8. Data Preparation and Machine Learning Methodology
 9. Backend API Implementation
 10. Frontend and Database Management
 11. How to Run the Project
 12. Testing and Results
 13. Limitations and Future Improvements
 14. Team Work and Learning Outcomes
 15. Conclusion
 16. References
-
- Appendix A. Main Execution Commands
- Appendix B. API Request Example
- Appendix C. Execution Screenshots and Dashboard Preview

1. Abstract

In this project, we built a local Python application that predicts whether a water sample is drinkable or not drinkable using physicochemical measurements from a CSV dataset. The work was developed as a group project for the Python module. Instead of only training a model, we tried to create a complete workflow that includes data preprocessing, machine learning training, a FastAPI backend, a Streamlit interface, and a SQLite database for saving prediction history.

The dataset contains water quality measurements such as pH, hardness, solids, chloramines, sulfate, conductivity, organic carbon, trihalomethanes, and turbidity. These features are used to predict the target variable called Potability. Missing values are handled using mean imputation, the input features are standardized, and a Random Forest classifier is trained to learn the relation between the measurements and the final class.

The final application is divided into two main parts. The backend is implemented with FastAPI and exposes a /predict endpoint. The frontend is implemented with Streamlit and allows the user to enter measurements manually or load a row from a CSV file. The prediction result and confidence score are displayed to the user, and the operation can be saved in a SQLite database. The project files are shared on GitHub for submission and review.

2. Introduction

We chose this topic because water quality prediction is easy to understand and also useful as a practical example of classification with machine learning. In real situations, water analysis requires laboratory testing and expert interpretation. However, in this project, our goal was not to replace laboratory analysis, but to show how machine learning can be used as a support tool to classify water samples based on numerical measurements.

The project was developed in Python because it offers useful libraries for data processing, machine learning, API development, and interface creation. Through this work, we practiced several important steps in a real software project, such as organizing files, preparing the dataset, training a model, saving the model and scaler, creating a backend API, connecting it with a frontend interface, and sharing the final project on GitHub.

In our project, we focused on building a working local prototype rather than only training a machine learning model. Our objective was to understand the complete workflow, from loading and preprocessing the dataset to displaying the prediction result in a web dashboard. This made the project more practical for us because we could see how Python can be used not only for data analysis, but also for building a complete application that can be tested locally through a browser.

3. Problem Statement and Objectives

The main problem is to predict whether a given water sample is potable based on numerical water quality indicators. The prediction must be returned in a simple form so that a non-technical user can understand the result. The system therefore has to process the input, apply the trained model, and return a clear label: Drinkable or Not Drinkable.

The project objectives are:

- Prepare and clean the water potability dataset.
- Train a machine learning model able to classify water samples.
- Save the trained model and scaler for later use.
- Build a FastAPI backend that receives water measurements and returns a prediction.
- Build a Streamlit interface that allows the user to enter or load water quality values.
- Store prediction history in a local SQLite database.

- Share the complete project source code and related files on GitHub.

4. Dataset Description

The project uses the file `water_potability.csv`. It contains numerical measurements describing the quality of water samples. In the uploaded repository, the file has about 2,871 data records plus the header row. The target column is `Potability`, where the model learns to classify samples into potable and non-potable water.

Column	Description
ph	Acidity/basicity level of the water sample.
Hardness	Concentration of minerals related to calcium and magnesium.
Solids	Total dissolved solids measured in the sample.
Chloramines	Amount of chloramines used in water treatment.
Sulfate	Sulfate concentration in the water sample.
Conductivity	Electrical conductivity, related to dissolved ions.
Organic_carbon	Amount of organic carbon present in the sample.
Trihalomethanes	Chemical compounds that may appear after disinfection.
Turbidity	Cloudiness of the water sample.
Potability	Target variable: 1 for potable water and 0 for non-potable water.

Some columns contain missing values. The training scripts handle this issue by replacing missing values with the mean value of each column. This choice is simple and suitable for a first machine learning project because it keeps all rows available for training instead of removing many samples.

5. Tools and Technologies

Tool / Library	Role in the Project
Python	Main programming language used for data processing, training, API, and frontend logic.
Pandas	Loading and manipulating the CSV dataset.
NumPy	Preparing numerical arrays for prediction.
Scikit-learn	Preprocessing, train/test split, Random Forest model, and accuracy evaluation.
FastAPI	Backend API used to expose the prediction endpoint.
Uvicorn	Local server used to run the FastAPI application.
Streamlit	Interactive frontend for entering water measurements and showing results.
SQLite	Local database used to save prediction history.
GitHub	Version control and project submission platform.

6. Project Repository and File Structure

The project repository is named `ai-water-potability-prediction` and is available on GitHub. The repository contains the Python source files, the trained model, the scaler, the dataset, the presentation file, and the frontend folder. This organization makes it possible for the professor to review the code and run the project locally.

File / Folder	Purpose
README.md	Basic repository documentation.
water_potability.csv	Dataset used for training and testing.
train.py	Main training script: cleans data, scales features, trains the Random Forest model, evaluates it, and saves artifacts.
model.py	Additional model training/testing script.
model.pkl	Saved trained classifier.
scaler.pkl	Saved StandardScaler used to transform new input values.
main.py	FastAPI backend application and /predict endpoint.
app.py	Streamlit interface and prediction history management.
water_quality.py	Small helper script for loading and displaying the dataset.
water_quality.db	SQLite database used for prediction history.
Frontend/	Additional frontend files.
Presentation.pptx	Project presentation file.
LICENSE	MIT license file.

https://github.com/ualide/ai-water-potability-prediction		
 ualide Add files via upload	9cf3dfc · now	 3 Commits
 Frontend	Add files via upload	12 hours ago
 LICENSE	Initial commit	12 hours ago
 Presentation.pptx	Add files via upload	now
 README.md	Initial commit	12 hours ago
 app.py	Add files via upload	12 hours ago
 main.py	Add files via upload	12 hours ago
 model.pkl	Add files via upload	12 hours ago
 model.py	Add files via upload	12 hours ago
 package-lock.json	Add files via upload	12 hours ago
 package.json	Add files via upload	12 hours ago
 scaler.pkl	Add files via upload	12 hours ago
 train.py	Add files via upload	12 hours ago
 water_potability.csv	Add files via upload	12 hours ago
 water_quality.db	Add files via upload	12 hours ago
 water_quality.py	Add files via upload	12 hours ago

Figure 1. GitHub repository structure provided for the project.

7. System Architecture

The application follows a simple client-server architecture. The user interacts with the Streamlit frontend. When the user clicks the analysis button, the frontend sends the entered values to the FastAPI backend through an HTTP POST request. The backend loads the saved scaler and model, transforms the input values, applies the Random Forest classifier, and returns the predicted class with a confidence value. The frontend displays the result and saves it in the SQLite history table.

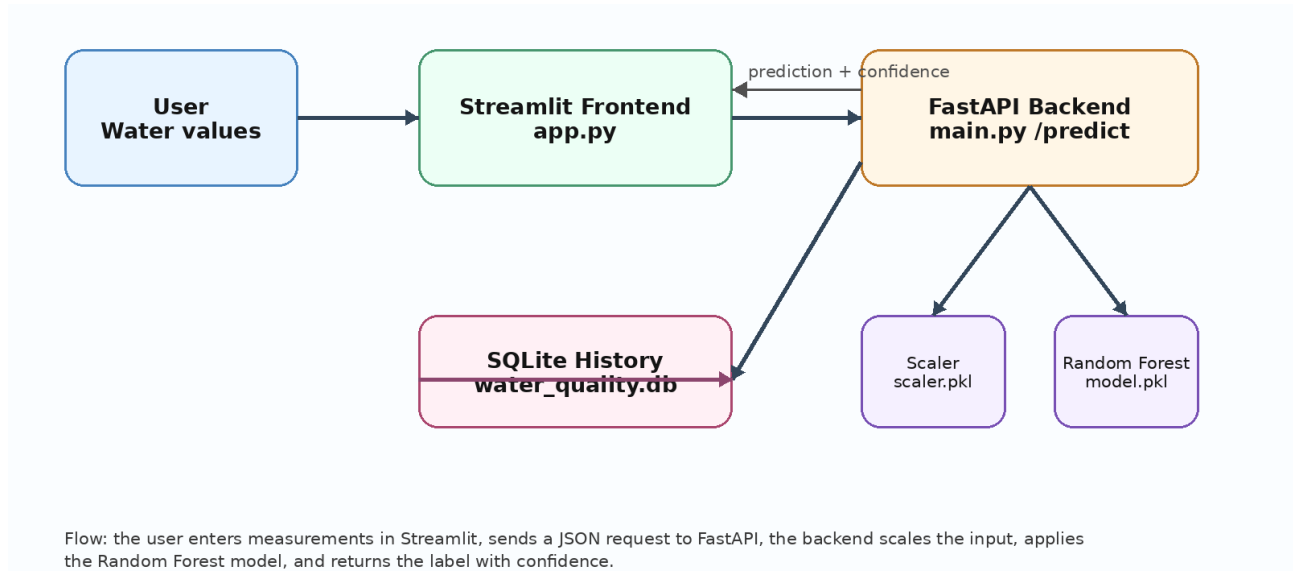


Figure 2. Main system architecture of the AI Water Potability Prediction application.

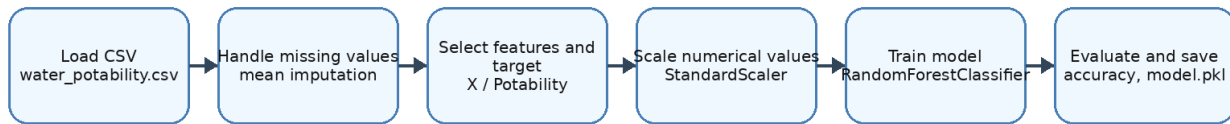
This separation between frontend and backend is useful because the model is not directly handled by the user interface. The API receives structured data, applies the same preprocessing used during training, and returns a response in JSON format. This makes the system easier to test and easier to improve later.

8. Data Preparation and Machine Learning Methodology

The machine learning part is implemented mainly in train.py. The dataset is first loaded using Pandas. For the missing data, we used a simple mean imputation method, where each empty value was filled using the average value of its column. The target variable Potability is separated from the input features. The numerical features are then scaled using StandardScaler so that the model receives values on a more balanced scale.

After preprocessing, the data is split into a training set and a test set. The training set is used to fit the model, while the test set is used to evaluate the predictions. The chosen model is RandomForestClassifier with 200 trees. A Random Forest is suitable for this project because it can handle nonlinear relationships between variables and is usually robust for tabular classification tasks.

Machine Learning Pipeline



The pipeline is implemented mainly in `train.py`: it prepares the dataset, trains the classifier, evaluates predictions, and saves the artifacts used by the API.

Figure 3. Machine learning pipeline implemented in the training script.

Step	Implementation
Missing values	Replaced using the mean of each column.
Feature scaling	StandardScaler is fitted on the input features.
Train/test split	The dataset is divided into training and testing parts.
Model	RandomForestClassifier with 200 estimators.
Evaluation	Accuracy is calculated using <code>accuracy_score</code> .
Saved artifacts	<code>model.pkl</code> and <code>scaler.pkl</code> are stored for later prediction through the API.

The trained model and scaler are saved as files so that the application does not need to retrain the model each time it starts. This is a common workflow in machine learning applications: training is performed once, then the trained artifacts are reused for prediction.

9. Backend API Implementation

The backend is implemented in `main.py` using FastAPI. The API loads `model.pkl` and `scaler.pkl` from the project directory. It also enables CORS so that the frontend can communicate with the backend during local execution. The main endpoint is `/predict`, and it receives the water measurements as a JSON object.

API Element	Description
Endpoint	<code>/predict</code>
HTTP method	POST
Input format	JSON object containing the nine water quality features.
Output format	JSON response containing prediction, label, and confidence.
Prediction labels	Drinkable if the predicted class is 1, Not Drinkable if the predicted class is 0.
Model files used	<code>model.pkl</code> and <code>scaler.pkl</code>

Before prediction, the backend organizes the received values into a NumPy array and applies the saved scaler. The scaled values are then passed to the trained classifier. The backend also uses `predict_proba` to estimate a confidence score, which is sent back to the frontend with the final label.

10. Frontend and Database Management

The frontend is implemented in app.py using Streamlit. The interface provides number input fields for the main water quality parameters. It also supports loading a CSV file and selecting a row index, which makes testing easier because the user can quickly load sample values instead of typing everything manually.

When the user clicks the analysis button, Streamlit sends the values to the backend API at <http://127.0.0.1:9000/predict>. If the API response is successful, the result is shown as Drinkable or Not Drinkable, and the confidence percentage is displayed using a metric component.

The application also manages a local SQLite database named water_quality.db. The database contains a table called predictions_history. Each saved row contains the input measurements, the prediction result, the confidence score, and the creation date. This feature is useful because it gives a simple history of previous tests performed through the interface.

Database Field Group	Purpose
ph, hardness, solids	Main physical/chemical input values.
chloramines, sulfate, conductivity	Treatment and dissolved ion indicators.
organic, trihalo, turbidity	Additional quality indicators used by the model.
result	Prediction label returned by the API.
confidence	Highest class probability returned by the model.
created_at	Timestamp automatically generated by SQLite.

11. How to Run the Project

The backend and the frontend must be launched separately. Both commands should be executed from the main project directory. The project directory used during development was:

```
cd "C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project"
```

First, the backend API is started using Uvicorn:

```
python -m uvicorn main:app --port 9000 --reload
```

This command runs the FastAPI backend locally on port 9000. After that, the Streamlit interface is launched using:

```
python -m streamlit run app.py
```

Once both parts are running, the user can open the Streamlit interface in the browser, enter the water quality values, and request a prediction through the backend API.

Recommended installation command before running the project:

```
pip install -r requirements.txt
```

12. Testing and Results

The project includes two levels of testing. The first level is model testing, which was performed by running the training script train.py. After preprocessing the dataset, splitting it into training and testing sets, and training the Random Forest classifier, the model obtained an accuracy of 66.96% on the test set.

This result shows that the model can make acceptable predictions for an educational prototype. However, the accuracy also shows that the model is not perfect and still needs further evaluation before being used in a real-world context. For a stronger evaluation, additional metrics such as precision, recall, F1-score, confusion matrix, and cross-validation could be added.

The second level is application testing. In this part, the backend was launched using FastAPI and Uvicorn, while the frontend was launched using Streamlit. The frontend successfully sent water quality values to the backend API, and the API returned a prediction label with a confidence score. The result was then displayed in the dashboard and saved in the SQLite database.

Test	Expected Result
Model training test	Run train.py and verify that the model is trained and saved without errors.
Artifact loading test	Start main.py through Uvicorn and confirm that model.pkl and scaler.pkl are loaded.
API prediction test	Send a JSON request to /predict and verify that prediction, label, and confidence are returned.
Frontend test	Run app.py with Streamlit, enter values, and check that the result appears in the interface.
Database test	After a prediction, verify that the result is saved in predictions_history.

The uploaded version includes the trained model and the scaler, so the application can perform predictions without retraining each time. Overall, the tests confirmed that the main workflow of the project works correctly: the model loads successfully, the API responds to prediction requests, the dashboard displays the result, and the prediction history is stored in the SQLite database.

It is important to mention that this type of project should be considered as an educational decision-support example. The prediction should not be used as a real public health decision without laboratory validation and expert review.

```
C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project>python train.py
Model Accuracy: 66.96%
```

Figure 4. Model accuracy obtained after running train.py. The Random Forest classifier achieved 66.96% accuracy on the test set.

13. Limitations and Future Improvements

Although the project works as a complete local prototype, some limitations remain. The dataset size is limited, and the mean imputation method is simple. The model also needs stronger evaluation using precision, recall, F1-score, confusion matrix, and cross-validation. These metrics would give a more complete view of performance, especially because potable and non-potable classes may not have the same importance in real situations.

Possible improvements include:

- Add a confusion matrix and classification report to the training script.
- Compare Random Forest with other models such as Logistic Regression, SVM, Gradient Boosting, or XGBoost.
- Use a clearer configuration file for model paths and API settings.
- Improve the README with installation steps, screenshots, and example API requests.
- Add form validation to avoid unrealistic input values.
- Deploy the backend and frontend online for easier demonstration.
- Add unit tests for preprocessing and API prediction.

14. Team Work and Learning Outcomes

Through this project, we had the opportunity to apply several concepts studied in the Python module in one complete application. The work was divided between the group members according to the main parts of the project, from data preparation to application testing and documentation.

Oualid Ouarrach was mainly responsible for the most technical part of the project. He worked on the general project organization, the integration between the trained machine learning model and the application, the FastAPI backend structure, the connection between the frontend and the prediction API, the GitHub repository preparation, and the final documentation. His work helped connect the different parts of the system into one functional application.

Mahmoud Charabara contributed to data preprocessing and machine learning training. He worked on preparing the dataset, handling missing values, training the Random Forest model, and testing the model accuracy.

Youness Eziddy contributed to the frontend part and application testing. He helped with the Streamlit dashboard, loading CSV values, testing predictions from the interface, and checking that the results were displayed correctly.

Imad Addioui worked on the SQLite database part and prediction history testing. He helped verify that prediction results, confidence values, and timestamps were saved correctly in the local database.

During the development process, we learned that a machine learning project is not only about training a model. It also requires preparing clean data, saving trained artifacts, creating a backend service, building a usable interface, testing the full workflow, and organizing the project files clearly.

Learning Area	What Was Practiced
Data preparation	Understanding CSV loading, missing values, and feature/target separation.
Machine learning	Training and saving a supervised classification model.
Backend development	Creating an API endpoint and returning JSON predictions.
Frontend development	Building an interactive interface with Streamlit.
Database use	Saving prediction history using SQLite.
Integration	Connecting the trained model, backend API, frontend dashboard, and SQLite database.
Collaboration	Sharing the project files and presentation through GitHub.

15. Conclusion

At the end of this project, we were able to build a functional Python application that starts from a CSV dataset and ends with a prediction displayed in a web interface. The group prepared the water quality data, handled missing values, trained a Random Forest classifier, saved the model and scaler, created a FastAPI backend, developed a Streamlit dashboard, and added SQLite prediction history.

This project helped us understand the full workflow of a machine learning application, not only the model training part. We also learned how to connect different components together: dataset, preprocessing, saved model, API, frontend, and database. The final application works locally and allows the user to enter or load water measurements, send them to the backend, and receive a clear result with a confidence score.

The project remains an educational prototype, and it can still be improved with stronger evaluation metrics, better input validation, more model comparisons, and online deployment. However, it shows that Python can be used to build a complete and practical application combining machine learning, backend development, frontend design, and database management.

16. References

1. Python Software Foundation. Python Documentation. <https://docs.python.org/>
2. Pandas Development Team. Pandas Documentation. <https://pandas.pydata.org/docs/>
3. Scikit-learn Developers. Scikit-learn User Guide. https://scikit-learn.org/stable/user_guide.html
4. FastAPI Documentation. <https://fastapi.tiangolo.com/>
5. Streamlit Documentation. <https://docs.streamlit.io/>
6. GitHub Repository: ualide/ai-water-potability-prediction. <https://github.com/ualide/ai-water-potability-prediction>
7. Project dataset: water_potability.csv uploaded in the GitHub repository.

Appendix A. Main Execution Commands

Backend command:

```
cd "C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project"
python -m uvicorn main:app --port 9000 --reload
```

Frontend command:

```
cd "C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project"
python -m streamlit run app.py
```

Appendix B. API Request Example

Example JSON request sent to the /predict endpoint:

```
{
  "ph": 7.0,
  "Hardness": 150.0,
  "Solids": 20000.0,
  "Chloramines": 7.0,
  "Sulfate": 300.0,
  "Conductivity": 400.0,
  "Organic_carbon": 10.0,
  "Trihalomethanes": 60.0,
  "Turbidity": 4.0
}
```

Example response structure:

```
{
  "prediction": 1,
  "label": "Drinkable",
  "confidence": 0.82
}
```

Appendix C. Execution Screenshots and Application Preview

This appendix documents the local execution of the project. The screenshots show that the FastAPI backend, the Streamlit frontend, the prediction dashboard, and the SQLite prediction history are all working as expected. They also provide visual evidence that the system can load a CSV file, analyze a selected row, return a prediction with confidence, and store previous predictions.

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\PC>cd "C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project"

C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project>python -m uvicorn main:app --port 9000 --
reload
INFO:      Will watch for changes in these directories: ['C:\\Users\\PC\\Desktop\\M1 - IISE - S2\\Python\\Projet Python\\
\water-quality-project']
INFO:      Uvicorn running on http://127.0.0.1:9000 (Press CTRL+C to quit)
INFO:      Started reloader process [24452] using WatchFiles
✓ API: Model and Scaler are Ready!
INFO:      Started server process [4220]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Figure C1. FastAPI backend launched with Uvicorn on port 9000. The terminal confirms that the model and scaler are ready and that the application startup is complete.

```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. Alle Rechte vorbehalten.

C:\Users\PC>cd "C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project"

C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project>python -m streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.178.120:8501
```

Figure C2. Streamlit frontend launched successfully. The application is available locally through the browser at localhost:8501.

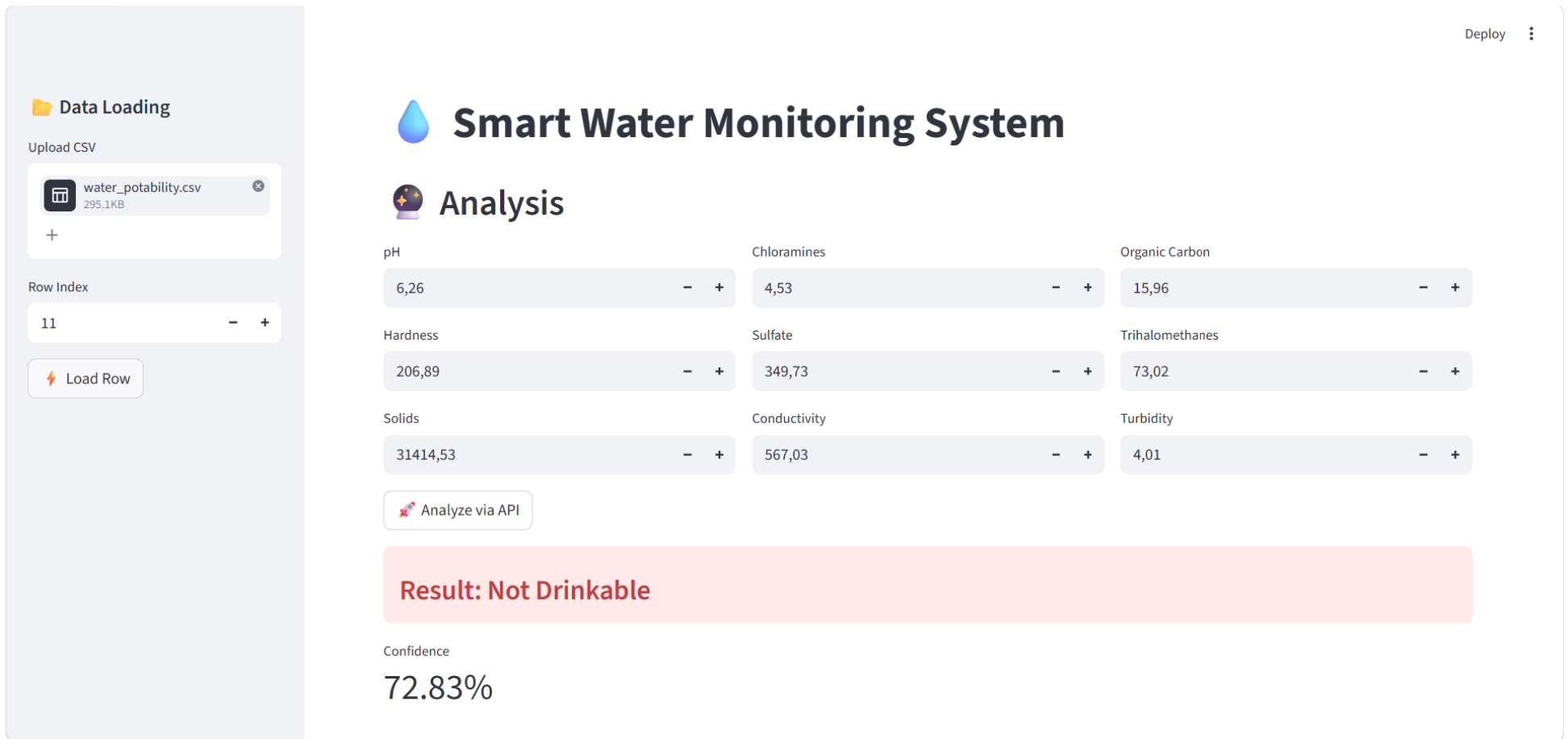
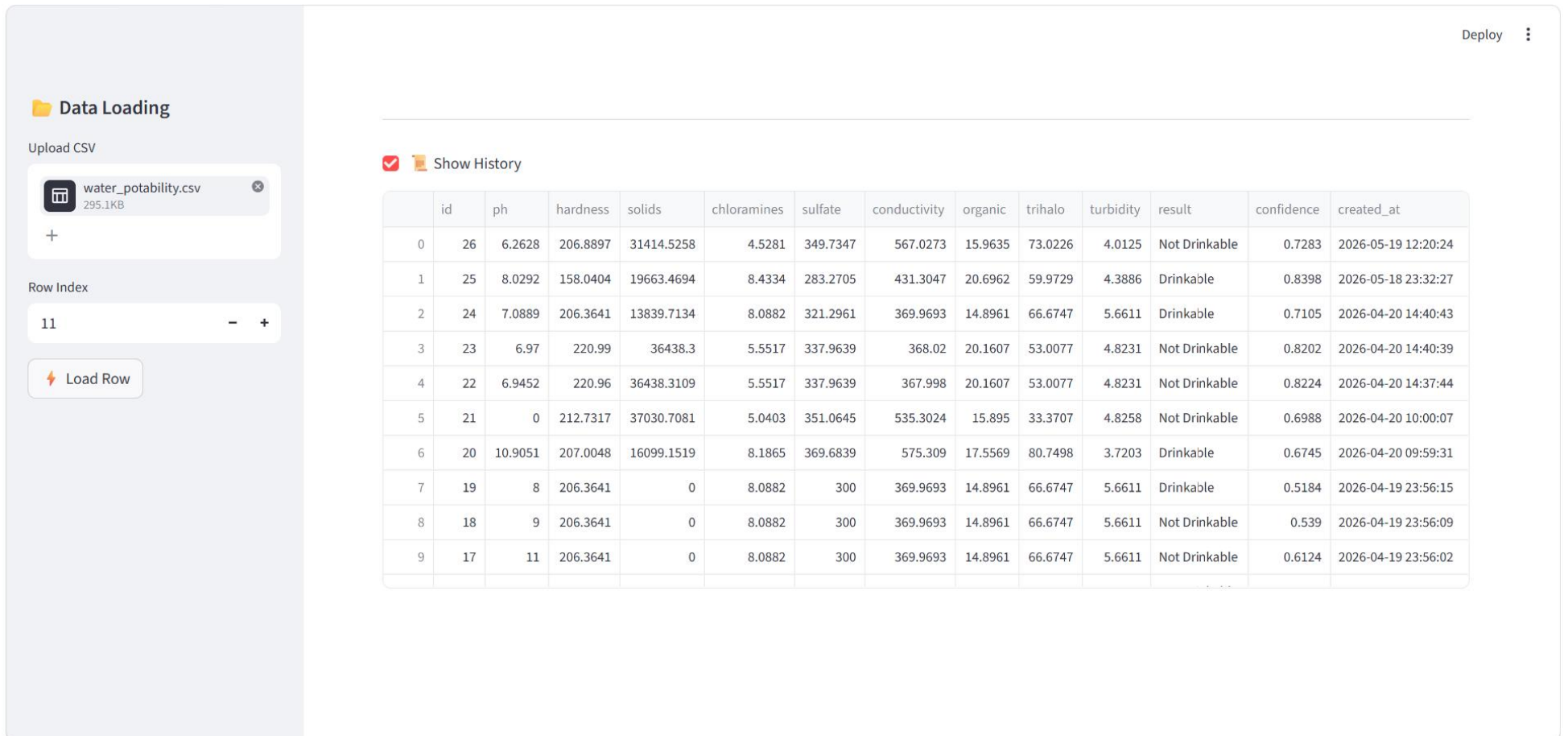


Figure C3. Main dashboard of the Smart Water Monitoring System. The CSV file is loaded, row 11 is selected, and the system returns Not Drinkable with a confidence of 72.83%.



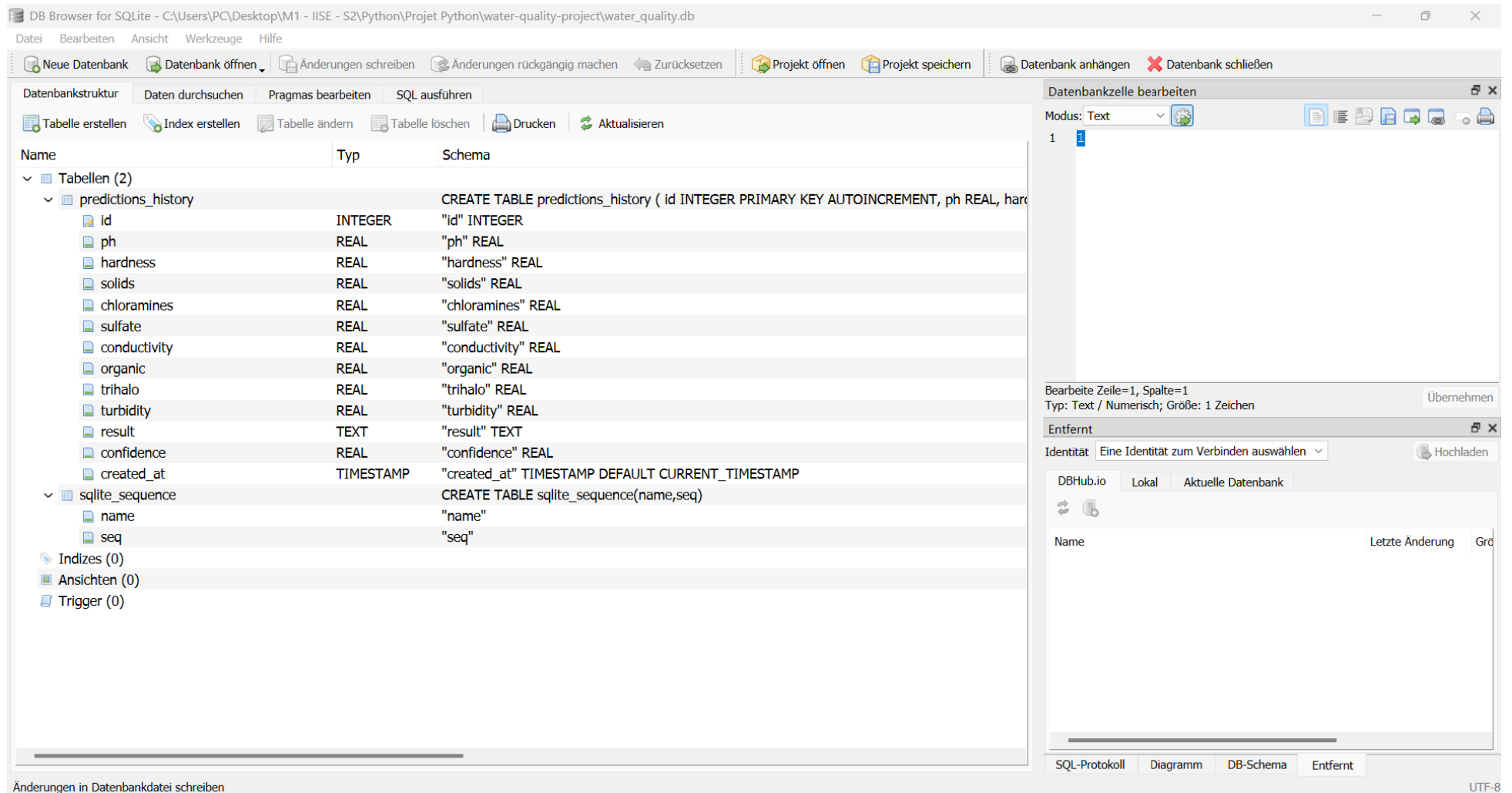


Figure C5. SQLite database schema showing the predictions_history table and its fields, including input parameters, result, confidence, and creation timestamp.

DB Browser for SQLite - C:\Users\PC\Desktop\M1 - IISE - S2\Python\Projet Python\water-quality-project\water_quality.db

Datei Bearbeiten Ansicht Werkzeuge Hilfe

Neue Datenbank Datenbank öffnen Änderungen schreiben Änderungen rückgängig machen Zurücksetzen Projekt öffnen Projekt speichern Datenbank anhängen ✗ Datenbank schließen

Datenbankstruktur Daten durchsuchen **Pragmas bearbeiten** SQL ausführen

Tabelle: predictions_history In allen Spalten filtern

	id	ph	hardness	solids	chloramines	sulfate	conductivity	organic	trihalo	turbidity	resu
	Filtern	Filtern	Filtern	Filtern	Filtern	Filtern	Filtern	Filtern	Filtern	Filtern	Filtern
1	1	7.078659419	232.5818483	26177.78951	3.914353184	334.1611267	380.4599178	9.633631144	41.95193959	3.696640141	Not Dri
2	2	4.197583589	189.4736767	31292.1277	7.105204272	0.0	370.7714128	11.98640181	50.32283042	5.206480177	Not Dri
3	3	7.088940642	206.3641219	13839.71343	8.08824197	321.2961415	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
4	4	7.0	150.0	20000.0	7.0	300.0	400.0	10.0	60.0	4.0	Not Dri
5	5	7.0	150.0	20000.0	7.0	300.0	400.0	10.0	60.0	4.0	Not Dri
6	6	7.0	150.0	20000.0	7.0	300.0	400.0	10.0	60.0	4.0	Not Dri
7	7	6.492638846	172.8639603	14625.56136	7.7365695	349.4040572	652.5375916	10.21205821	56.94972417	4.667770161	Not Dri
8	8	7.814524321	180.3473084	17516.69936	7.489006438	352.2770291	370.0113729	14.28931979	80.88242514	2.901699556	Not Dri
9	9	8.15121531	219.56005	13699.68071	3.804428299	0.0	460.528736	14.95129501	79.63426081	3.523980506	Drinkab
10	10	8.05	219.56005	13699.68071	3.87	0.0	460.528736	14.95129501	79.63426081	3.523980506	Drinkab
11	11	7.12	206.3641219	13839.71343	8.08824197	321.2961415	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
12	12	7.12	206.3641219	13839.71343	8.08824197	321.2961415	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
13	13	7.12	206.3641219	13839.71343	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
14	14	7.12	206.3641219	13839.71343	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
15	15	7.12	206.3641219	0.0	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
16	16	19.0	206.3641219	0.0	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Not Dri
17	17	11.0	206.3641219	0.0	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Not Dri
18	18	9.0	206.3641219	0.0	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Not Dri
19	19	8.0	206.3641219	0.0	8.08824197	300.0	369.9693333	14.89609216	66.67465815	5.661104287	Drinkab
20	20	10.90507641	207.0048366	16099.1519	8.186478946	369.6838669	575.3090369	17.55687962	80.74984922	3.720264379	Drinkab

1 - 20 von 26 Springe zu: 1

Änderungen in Datenbankdatei schreiben

Datenbankzelle bearbeiten

Modus: Text

Bearbeite Zeile=1, Spalte=1
Typ: Text / Numerisch; Größe: 1 Zeichen Übernehmen

Entfernt ✗

Identität Eine Identität zum Verbinden auswählen Hochladen

DBHub.io Lokal Aktuelle Datenbank

Name Letzte Änderung Grd

SQL-Protokoll Diagramm DB-Schema Entfernt

UTF-8

Figure C6. SQLite database data view showing stored prediction records in the predictions_history table.